Prism evaluation 5 of 8

Risk rule engine

Core: risk module, ruleset risk-rules-2 · Run 08 Nov 2026, 17:00 UTC · **All targets met**

What is measured

Whether each deterministic rule fires on the claim built to trigger it, end to end: documents read by the real model, consolidated, and screened as in production. A rule can only fire on what extraction reads, so this measures the whole chain, not the rule code alone (which core's unit tests cover rule by rule).

Data

25 synthetic claims loaded by prism.py demo-prepare: 16 built to trigger specific rules (25 planted signals across 21 rules), 8 clean claims that should raise nothing, and one waiting for a document.

Results

Measure	Result	Target	
Planted signals detected	**25 / 25 (100.0%)**	100%	**Met**
Signals raised beyond those planted	**0**	0	**Met**
Precision of raised signals	**100.0%**		
Clean claims with no indicator	**8 / 8 (100.0%)**		
Rules exercised	**21 of 22**		
Bands: low, medium, high	**13, 9, 2**		

Rule by rule

Rule	Claims built to raise it	Raised on them	Raised elsewhere
CON-01	2	2	0
CON-02	1	1	0
CON-03	3	3	0
CON-04	2	2	0
CON-05	1	1	0
CON-06	1	1	0
DOC-01	1	1	0
DOC-02	1	1	0
DOC-03	1	1	0
DUP-01	1	1	0
DUP-02	1	1	0
DUP-03	1	1	0
DUP-04	1	1	0
DUP-05	1	1	0
FIN-01	1	1	0
FIN-02	1	1	0
FIN-03	1	1	0
FIN-04	1	1	0
FIN-05	1	1	0
FIN-06	1	1	0
MIS-02	1	1	0

Observations

• MIS-01 (a required document missing) is not exercised here by design: such a claim waits 14 days for the document before it is screened. Core's unit tests cover it.
• Building this set found three mistakes in the data, none in Prism: hand-typed amounts that did not match their bills (FIN-03 reported them), a bill whose summary line was read as an item, and expectations that ignored DUP-01 taking precedence over DUP-02 and DUP-03 for the same file (D-096).

Limitations

• Documents written for the purpose. Each rule is exercised by one or two claims, so this shows the chain works, not how often rules fire on real claims.

Reproduce

```
python -m uv run --project services/ai python scripts/prism.py demo-prepare
```